Recurrent dynamics, from one equation

A theory guide to Lecture 6 and the PN3 ramp practice

How to read this

Everything in Lecture 6 comes from one fact about synapses. Section 1 states that fact. Each later section either chooses what to feed the synapse to get a behaviour you want, or works out what happens when that input is slightly wrong. Read the sections in order the first time. Afterwards, Section 12 is a one-page summary to revise from.

The ramp sections each part supports:

This guide	Ramp practice
1–2 The synapse and the loop	8, 9
3 Linear feedback	10
4 Phase portraits	11
5 Designing a system: $A' = \tau A + I$	12
6 The ideal integral	13
7 Why integrators drift	14
8 Unequal synapses	15
9 Leaky integrators	16
10 Oscillators	17
11 Nonlinear systems and the controlled integrator	18, 19

1 The one fact: a synapse chases its input

Let $z(t)$ be whatever arrives at a synapse, and $y = h * z$ what comes out, with the exponential synapse $h(t) = \frac{1}{\tau} e^{-t/\tau}$. Then

$$\boxed{\tau\,\dot{y} = z - y}$$

Why. $y(t) = \int_{-\infty}^{t} \frac{1}{\tau} e^{-(t-t')/\tau} z(t')\,dt'$. Differentiating, the upper limit contributes $\frac{1}{\tau} z(t)$, and differentiating the exponential contributes $-\frac{1}{\tau} y(t)$. So $\dot{y} = (z - y)/\tau$.

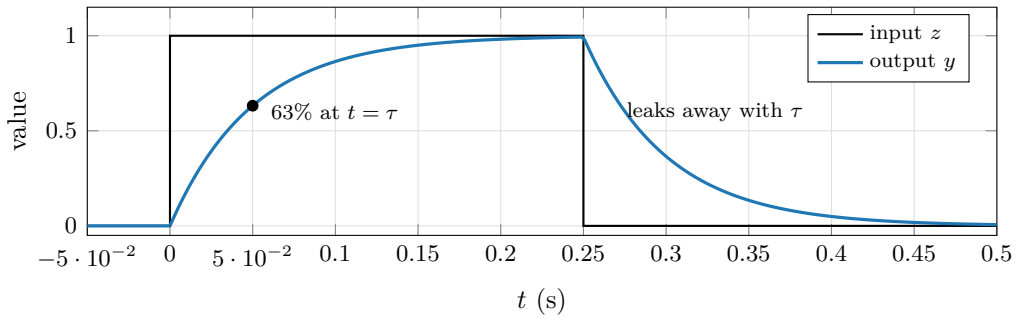
The Laplace route gives the same thing: $H(s) = \frac{1}{1+s\tau}$, so $Y(1+s\tau) = Z$, and $s \to d/dt$ gives $\tau\dot{y} = z - y$.

How to picture it

The output moves toward the input at a speed proportional to how far apart they are. If z jumps to a new value, y covers 63% of the gap in one τ and 95% in 3τ. If z is removed, y decays back to zero with time constant τ.

Two names for the same behaviour: a *low-pass filter* (it smooths fast changes) and a *leaky integrator* (it accumulates its input, $\dot{y} \propto z$, but leaks away its own value, the $-y$ term).

A synapse ($\tau = 50$ ms) chasing an input that turns on, then off



2 The neural loop

A population represents x. It receives $g(u)$ from an input connection and $f(x)$ from a connection onto itself, both through the same synapse. So the synapse's input is $z = g(u) + f(x)$, and its output is x. Put those into Section 1:

$$\tau \dot{x} = \underbrace{g(u) + f(x)}_{z \text{ (what we feed it)}} - \underbrace{x}_{\text{the leak}}$$

Key idea

This is the *neural loop equation*. Every question in Lecture 6 starts by writing it down, deciding what z is, and simplifying. The $-x$ is always present: it is the synapse leaking. A recurrent connection is the only way to put something back in to oppose it.

Why a feed-forward network forgets. Without the recurrent connection, $f = 0$ and $\tau \dot{x} = g(u) - x$. When the input stops, $\tau \dot{x} = -x$, so x decays to zero within a few τ (tens of milliseconds). Nothing is fed back to oppose the leak.

Why the algebraic guess fails. Writing $x = g(u) + f(x)$ (as Lecture 6 first tries) is only the condition $\dot{x} = 0$, the place where the state would sit still. It says nothing about how fast the state gets there, or whether it gets there at all. For $f(x) = x + 1$ it even reduces to $0 = u + 1$, which is nonsense as a statement about x. The differential equation is the real description.

3 Linear feedback: forget, remember, or explode

Take $g(u) = u$ and $f(x) = kx$. The loop gives $\tau \dot{x} = u - (1 - k)x$. The quantity that matters is $1 - k$, *the leak that is left over*: the synapse leaks x, and the feedback puts kx back. For a unit step from $x(0) = 0$:

$$x(t) = \frac{1}{1-k}\left(1 - e^{-(1-k)t/\tau}\right), \qquad \text{steady state } \frac{1}{1-k}, \quad \text{time constant } \frac{\tau}{1-k}.$$

Feedback gain	Left-over leak $1 - k$	Behaviour
$k < 1$	positive	settles at $u/(1-k)$ with time constant $\tau/(1-k)$: **forgets**
$k = 1$	zero	$\tau \dot{x} = u$: an integrator, **remembers**
$k > 1$	negative	grows exponentially until the neurons saturate: **explodes**

Quick check. Plug in $k = -1$: steady state $\frac{1}{2}$, time constant $\tau/2$. This matches Lecture 6's "converges to $u/2$". The feedback $-x$ doubles the leak, so the loop is twice as fast as the synapse alone.

The $k = 1$ case is the limit of the formula: $e^{-\epsilon t/\tau} \approx 1 - \epsilon t/\tau$ for small $\epsilon = 1 - k$, so $x \to t/\tau$. The steady state and the time constant both go to infinity together.

4 Phase portraits: reading behaviour off a graph

When f is nonlinear you usually cannot solve for $x(t)$. You don't need to. In one dimension, $\dot{x} = \phi(x)$ already tells you which way the state moves at every x.

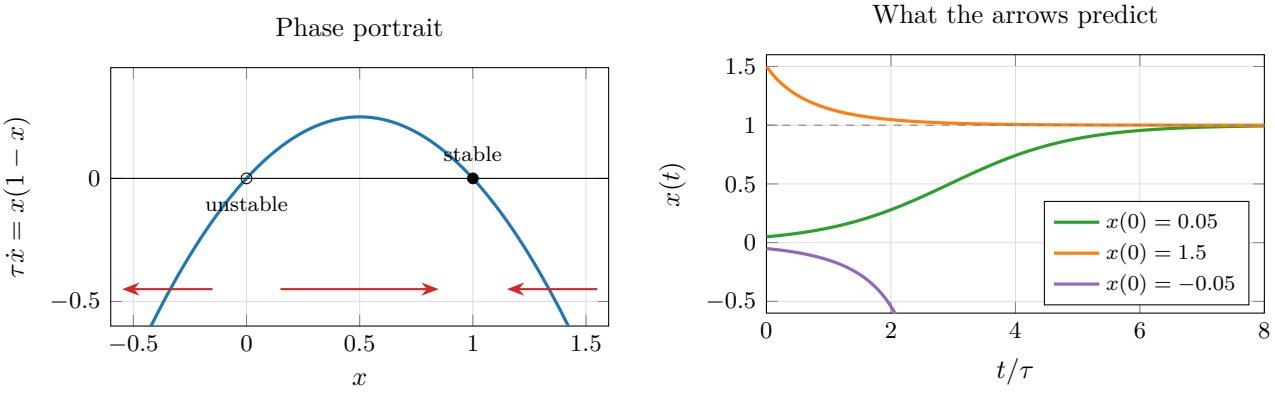
The five steps

1. **Write the loop.** $\phi(x) = \frac{1}{\tau}(u + f(x) - x)$ with u held at a number. Don't drop the $-x$.
2. **Equilibria** are where $\phi(x) = 0$: the x-intercepts. There the state stops changing.
3. **Directions.** Where $\phi > 0$, x increases (moves right). Where $\phi < 0$, x decreases (moves left). Draw arrows on the x-axis.
4. **Stability.** Arrows pointing in from both sides: stable. Pointing away: unstable. Shortcut: $\phi'(x^*) < 0$ is stable and $\phi'(x^*) > 0$ is unstable. If $\phi'(x^*) = 0$, the shortcut cannot decide, so use the arrows.
5. **Behaviour.** From any start, follow the arrow to the next stable equilibrium. If there is none in that direction, the state runs off until the neurons saturate.

Common trap

The state does not slide along the curve, and the bottom of the curve is not an equilibrium. The curve is the state's *velocity*. Its height at each x says how fast and in which direction x is changing. The state itself only moves along the x-axis.

Example (not from the ramp): $f(x) = 2x - x^2$, $u = 0$, so $\tau\phi = x(1 - x)$. Equilibria at 0 and 1; $\tau\phi' = 1 - 2x$ is $+1$ at 0 (unstable) and -1 at 1 (stable).



Why the slope decides stability. Near an equilibrium the curve is almost a straight line, $\phi(x) \approx \phi'(x^*)(x - x^*)$. A small nudge $\delta = x - x^*$ then obeys $\dot{\delta} = \phi'(x^*)\,\delta$, so $\delta(t) = \delta_0 e^{\phi'(x^*)t}$: it shrinks when

the slope is negative and grows when it is positive. This is the one-dimensional version of the lecture's rule about the eigenvalues of the Jacobian.

Why only stable equilibria matter in neurons. Spike noise is constantly nudging the state, so a network can only rest at a stable equilibrium. An unstable one is a dividing line between outcomes, not a resting place.

Two ideas that tests like.

- *The input shifts the curve.* u adds a constant to ϕ, moving the curve up or down. Two equilibria can move together, merge, and disappear. After that, the state has nowhere to stop. For Lecture 6's $f(x) = x^2$ this happens at $u = 1/4$.
- *Two stable equilibria with an unstable one between them make a memory.* Where the state ends up depends on which side of the unstable point it started, not on the current input. The network stores one bit.

5 Designing a system: $A' = \tau A + I$, $B' = \tau B$

Now turn it around. Given a target $\dot{x} = Ax + Bu$, choose what to feed the synapse.

The problem. An ideal implementation computes $Ax+Bu$ and passes it through a perfect integrator. Neurons have no perfect integrator, only the leaky synapse of Section 1. Feeding it $Ax + Bu$ directly gives the wrong system: the leak is added, and everything is scaled by $1/\tau$.

The idea. Feed the synapse $z = A'x + B'u$ instead: $A'x$ from a recurrent connection and $B'u$ from the input connection. Then choose A' and B' so the loop behaves exactly like the target.

The derivation. The loop equation with this z:

$$\tau\dot{x} = A'x + B'u - x.$$

To compare with Ax, the two x terms must be combined into one matrix times x. You cannot write $(A' - 1)x$, because A' is a matrix and 1 is a number. Write the lone x as Ix, since the identity leaves every vector unchanged:

$$A'x - x = A'x - Ix = (A' - I)x \quad \Longrightarrow \quad \dot{x} = \frac{A' - I}{\tau}\, x + \frac{B'}{\tau}\, u.$$

Matching coefficients with $\dot{x} = Ax + Bu$:

$$\boxed{A' = \tau A + I, \qquad B' = \tau B}$$

In 2-D, $A'x - x$ subtracts x_1 from the first row and x_2 from the second, which is why the -1's land only on the diagonal: each component leaks only from itself. In 1-D, I is just the number 1.

Piece	What it does
$+I$	Feeds the current state back to cancel the synapse's leak. It is the $-x$ term carried through the algebra.
$\times\tau$	A and B are rates (units of 1/s). The synapse moves at (input − state)/τ per second, so to move at rate $Ax + Bu$ its input must be τ times larger. It also makes A', B' dimensionless, as a Nengo `transform` needs.
which τ	The synapse on the *recurrent* connection.

In Nengo, the input connection is `Connection(u, x, transform=B', synapse=tau)` and the recurrent one is `Connection(x, x, transform=A', synapse=tau)`.

Worked example (not from the ramp): $\dot{x} = -2x + 4u$ with $\tau = 0.1$ s. Then $A' = 0.1(-2) + 1 = 0.8$ and $B' = 0.4$.

- *Steady-state check.* Ideal: $\dot{x} = 0$ gives $x = 2u$. Neural: at rest the synapse's output equals its input, $x = 0.8x + 0.4u$, so $x = 2u$. They match.
- *Speed check.* $\tau_{\text{eff}} = \tau/(1 - A') = 0.1/0.2 = 0.5$ s, which equals $1/|A|$. They match.

Reading A' backwards. In 1-D, $A = (A' - 1)/\tau$. So $A' = 1$ is a perfect memory, $A' < 1$ leaks with $\tau_{\text{eff}} = \tau/(1 - A')$, and $A' > 1$ explodes. This is Section 3 again with $k = A'$.

The network can be faster than its synapses. A low-pass filter with τ_d shorter than τ needs $A' = 1 - \tau/\tau_d < 0$. Strong negative feedback plus a large input gain speeds the loop up beyond any single synapse.

6 The ideal answer of an integrator

For $\dot{x} = u$ from $x(0) = 0$, the ideal state is the running area under the input, $x(t) = \int_0^t u(t')\,dt'$. Three rules make it easy to sketch:

- The **slope** of x equals the current value of u.
- When $u = 0$, x **holds**. It does not fall back to zero; that is the memory.
- Each piece **starts where the previous one ended**. There is no "$+C$".

The process. For each region, split the integral at the switching times; only stretches with $u \neq 0$ contribute. Example: $u = 3$ on $0.5 \leq t < 1.5$, else 0.

$$x(t) = \begin{cases} 0, & t < 0.5, \\ \int_{0.5}^{t} 3\,dt' = 3(t - 0.5), & 0.5 \leq t < 1.5, \\ \int_{0.5}^{1.5} 3\,dt' = 3, & t \geq 1.5. \end{cases}$$

Check continuity at every switch (an integral cannot jump), and check that the final value is the pulse's area, height times width.

Common trap

The radius. The ideal integral may exceed the radius, but the neural state cannot. Once the neurons saturate, the extra area is lost. When the input reverses, the state comes down by the full integral starting from the clipped value, so it ends up permanently offset below the ideal. For $u = 3\sin 3t$ with radius 1, the ideal is $1 - \cos 3t \in [0, 2]$; after the first clip the neural state follows about $-\cos 3t$.

7 Why a neural integrator drifts

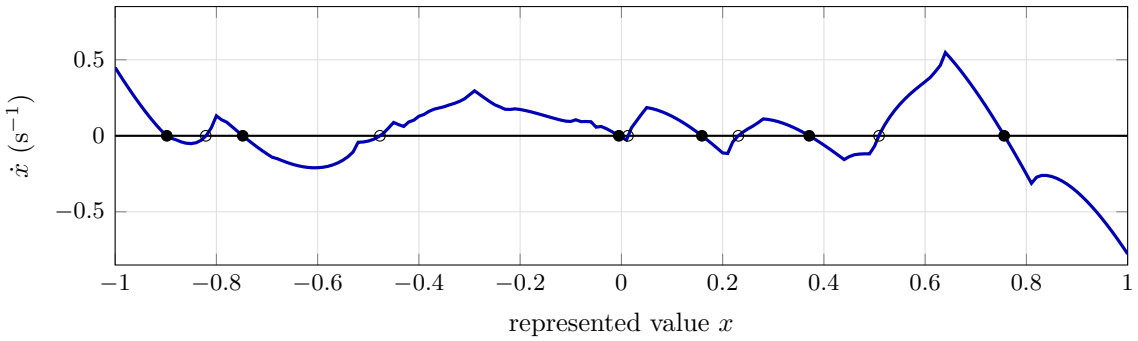
For an integrator the recurrent connection should feed back exactly x. Neurons only approximate it: the decoder gives $\hat{f}(x) = x + \text{error}(x)$. Put that into the loop with no input:

$$\tau\dot{x} = \hat{f}(x) - x = \text{error}(x).$$

Key idea

The integrator's velocity is its own decoding error. Plotting error$(x)/\tau$ against x gives a Section 4 phase portrait: a wiggly curve near zero with many small stable and unstable zeros. With no input, the state slides to the nearest stable zero.

Measured in Nengo: 30-neuron rate integrator, $\tau = 0.1$ s, no input



Error source	Effect on the loop	What you see
Static distortion (the decoder's fit is not perfect)	The fixed wiggly error(x) above	A smooth drift to the same place on every run
Spike noise (z fluctuates spike by spike)	Random kicks to the state	A random walk: each run ends somewhere different

Rate mode removes the spike noise but keeps the distortion. More neurons shrink both (distortion roughly as $1/n$, noise roughly as $1/\sqrt{n}$ in RMSE), so the drift slows, but it never stops completely.

8 Unequal input and recurrent synapses

PN3 uses a fast synapse on the input ($\tau_{\text{in}} = 5$ ms) and a slow one on the recurrence ($\tau = 50$ ms), and computes $A' = 1$, $B' = \tau$ with the slow one. What changes?

The intuitive picture: split the state in two

Each synapse has its own output, and the population represents their sum:

- p, the output of the **input** synapse. It chases τu quickly: $\tau_{\text{in}}\dot{p} = \tau u - p$.
- r, the output of the **recurrent** synapse. It chases x slowly: $\tau\dot{r} = x - r$.
- The state is $x = r + p$.

Substituting $x = r + p$ into the recurrent equation:

$$\tau\dot{r} = (r + p) - r = p \quad \Longrightarrow \quad \dot{r} = \frac{p}{\tau}.$$

So the recurrent synapse integrates whatever the input synapse delivers. The input synapse is fast, so after a few τ_{in}, $p \approx \tau u$, and then $\dot{r} \approx u$:

$$x = r + p \;\approx\; \underbrace{\int_0^t u}_{r\text{: stored}} \;+\; \underbrace{\tau u}_{p\text{: in transit}} .$$

How to picture it

With *equal* synapses, the input's push τu goes through the same slow synapse as the feedback, so it enters the state only gradually, at rate u. With a *fast* input synapse, the full push τu appears in the state almost at once, on top of the stored integral. The population represents both parts, so the decoded value includes both.

The exact result

The approximation $p \approx \tau u$ ignores the first few τ_{in}, and r really integrates the slightly delayed p. Accounting for that removes $\tau_{\text{in}} u$ from the lump:

$$\boxed{x(t) = \int_0^t u \;+\; (\tau - \tau_{\text{in}})\,(h_{\tau_{\text{in}}} * u)(t)}$$

where $h_{\tau_{\text{in}}} * u$ is just u smoothed by the fast synapse.

The Laplace derivation

Each connection gets its own filter:

$$X = \frac{1}{1 + s\tau}\, X + \frac{1}{1 + s\tau_{\text{in}}}\, \tau U.$$

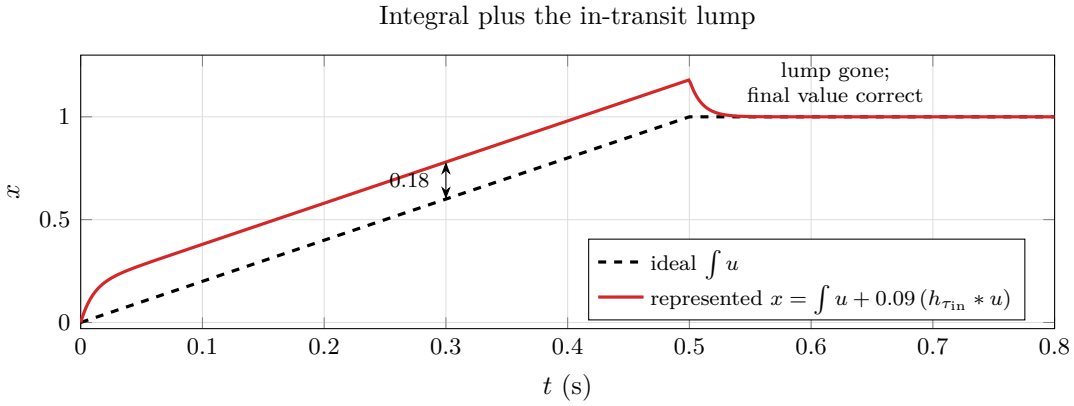
1. Collect the X terms: $X\left(1 - \frac{1}{1+s\tau}\right) = X\,\frac{s\tau}{1+s\tau}$.
2. Solve: $X\,\frac{s\tau}{1+s\tau} = \frac{\tau U}{1+s\tau_{\text{in}}}$, so $X = \frac{1+s\tau}{1+s\tau_{\text{in}}} \cdot \frac{U}{s}$.
3. Split the ratio: $\frac{1+s\tau}{1+s\tau_{\text{in}}} = 1 + \frac{s(\tau-\tau_{\text{in}})}{1+s\tau_{\text{in}}}$.
4. Multiply by U/s; in the second term the s cancels the $1/s$:

$$X = \underbrace{\frac{U}{s}}_{\int u} + \underbrace{(\tau - \tau_{\text{in}})\frac{U}{1 + s\tau_{\text{in}}}}_{(\tau-\tau_{\text{in}})(h_{\tau_{\text{in}}} * u)} .$$

Same result as the intuitive picture, reached by algebra.

What it looks like

Example (not from the ramp): $\tau = 0.1$ s, $\tau_{\text{in}} = 0.01$ s, $u = 2$ on $0 \le t < 0.5$, then 0. The lump is about $(0.1 - 0.01) \times 2 = 0.18$.



Sanity checks.

- $\tau_{\text{in}} = \tau$: the lump is zero, and the perfect integrator returns.
- The final value is unaffected: the lump exists only while $u \neq 0$.
- The lump is set by $\tau - \tau_{\text{in}} \approx \tau$, not by τ_{in}. A 5 ms input synapse does not just cause a 5 ms lag.

For PN3 §4. The integral stays correct, but while the input is on the trace sits about $(\tau - \tau_{\text{in}})u = 0.045 \times 0.9 \approx 0.04$ above the ideal, and it snaps back when the input stops. For ramp and sine inputs the lump follows u, so the trace looks slightly *ahead* of the ideal.

9 Leaky integrators, and why integrators are fragile

If the feedback is slightly less than x, $z = A'x$ with A' just below 1, then $\tau\dot{x} = -(1 - A')x$. Part of the leak survives, and the state decays with

$$\tau_{\text{eff}} = \frac{\tau}{1 - A'}.$$

Because this divides by a small number, memory is extremely sensitive to the feedback gain. With $\tau = 50$ ms: $A' = 0.99$ gives 5 s, $A' = 0.999$ gives 50 s, and $A' = 1.01$ grows until saturation. A 1% decoder error can change the memory time tenfold, which is why accurate neural integrators need many neurons.

Biology uses the leak deliberately. The horizontal eye-position circuit drifts back to centre with $\tau_{\text{eye}} \approx 70$ s, so it is modelled as $\dot{x} = -x/\tau_{\text{eye}} + v$, implemented with $A' = 1 - \tau/\tau_{\text{eye}}$ and $B' = \tau$.

10 The oscillator: a loop that rotates

With a 2-D state and $A = \begin{pmatrix} 0 & -\omega \\ \omega & 0 \end{pmatrix}$, the velocity Ax is always perpendicular to the position x. Anything whose velocity is always at right angles to where it is goes round in a circle, here at ω rad/s.

Read $A' = I + \tau A = \begin{pmatrix} 1 & -\tau\omega \\ \tau\omega & 1 \end{pmatrix}$ as: keep the current state (I), plus a small sideways nudge (τA). Each pass round the loop rotates the state a little.

- **It needs a kick.** At the origin $Ax = 0$, so nothing moves until an input pushes the state off zero.
- **The neurons set the amplitude.** The ideal oscillator keeps whatever amplitude the kick gave it. The neural one is bounded by the radius, and noise or distortion can make it shrink to zero.
- **High frequencies are harder.** A large $\tau\omega$ means each pass must produce a big rotation, so decoding errors and the synapse's approximations matter more. That is why a Nengo oscillator tracks worse as the frequency rises.

A damped oscillator adds $-\gamma$ on the diagonal of A; then A' has $1-\tau\gamma$ on its diagonal and the trajectory spirals inward.

11 Nonlinear systems, and the controlled integrator

Same recipe, with functions. Nothing in Section 5 needed z to be linear. For $\dot{x} = f(x) + g(u)$, feed the synapse

$$f'(x) = \tau f(x) + x, \qquad g'(u) = \tau g(u) :$$

the same two moves, scale the desired rate by τ and add x to cancel the leak. In Nengo the recurrent connection gets `function=`. If f multiplies state variables together (like $x_1 x_2$), one population must represent all of them together, as with xy in Test 2. The Lorenz attractor is one 3-D population for that reason, and it needs no kick because its origin is not an equilibrium.

The controlled integrator: a memory with a knob. Target $\dot{x} = c\,x + u$, where $c(t)$ is itself an input. Here c plays the role of the leak, and you can turn it while the network runs: $c = 0$ integrates, $c < 0$ forgets with time constant $1/|c|$, and $c > 0$ grows. Effectively the recurrent gain becomes $1 + \tau c$. The feedback $f'(x, c) = x + \tau c x$ contains the product cx, so one 2-D population represents (x, c) together. Only the x component is fed back; c enters dimension 2 through its own connection, with no recurrence on it.

12 One-page summary

Key idea

When a question looks unfamiliar: write $\tau\dot{x} = z - x$, decide what z is (everything fed into the synapse), and simplify. Then either read the behaviour off the sign of $\dot{x}$ (phase portrait), or match coefficients with the target system.

Topic	What z is	What to remember
Synapse	any input	$\tau\dot{y} = z - y$: chases its input, leaks with τ
Loop	$g(u) + f(x)$	$\tau\dot{x} = g(u) + f(x) - x$; $x = g + f$ is only the steady state
Linear feedback	$u + kx$	Left-over leak $1 - k$: forget, remember ($k = 1$), explode
Phase portrait	$u + f(x)$	Zeros are equilibria; downward crossing is stable; follow the arrows
Design	$A'x + B'u$	$A' = \tau A + I$, $B' = \tau B$, recurrent τ; check steady state and speed
Ideal integral	$x + \tau u$	Slope $= u$, holds when $u = 0$, no $+C$, watch the radius
Drift	$x + \text{error}(x)$	Distortion drifts the same way each run; noise random-walks
Unequal synapses	x (slow) $+\tau u$ (fast)	$\int u + (\tau - \tau_{\text{in}})(h_{\tau_{\text{in}}} * u)$; final value correct
Leaky integrator	$A'x$, $A' \approx 1$	$\tau_{\text{eff}} = \tau/(1 - A')$: extremely sensitive
Oscillator	$(I + \tau A)x$	Rotation; needs a kick; radius bounds the amplitude
Nonlinear	$\tau f(x) + x + \tau g(u)$	Same recipe with functions; products need one joint population
Controlled integrator	$x + \tau cx + \tau u$	Adjustable leak; 2-D population representing (x, c)